

# Nyaya.ai

Indian Legal AI Assistant

Next.js App Router • Supabase Auth & Postgres • Puter.js AI • TypeScript

Project Documentation — Prototype / Active Development

Runtime

**Next.js App Router**

Database

**Supabase Postgres**

AI Engine

**Puter.js (browser)**

Auth

**Supabase Auth + RLS**

Language

**TypeScript 5+**

Status

**Prototype / Active Dev**

This document was generated from the active root workspace at nyaya-ai-main/. The nested nyayaAI-main/ folder is a secondary snapshot and is not the primary reference.

## Table of Contents

| #  | Section               | Topic                                        |
|----|-----------------------|----------------------------------------------|
| 1  | Project Overview      | Purpose, users, status, mindmap              |
| 2  | Architecture & Design | System diagram, data flow, sequence          |
| 3  | Tech Stack            | Languages, frameworks, dependencies          |
| 4  | Project Structure     | Folders, modules, naming conventions         |
| 5  | Setup & Installation  | Prerequisites, env vars, setup flow          |
| 6  | Configuration         | Config files, settings, environment behavior |
| 7  | Database Schema       | Tables, relationships, ER diagram            |
| 8  | API Reference         | Endpoints, auth, request/response            |
| 9  | Key Algorithms        | Chat, documents, bookmarks, filtering        |
| 10 | Testing               | Strategy, coverage, gaps                     |
| 11 | Deployment            | Build, hosting, rollback                     |
| 12 | Known Issues          | Bugs, tech debt, impact chart                |
| 13 | Contributing          | Branching, commits, PR process               |
| 14 | Changelog             | Version timeline                             |
| 15 | Glossary              | Terms and naming patterns                    |

1

Project Overview

Nyaya.ai is a legal-assistance web application focused on Indian law. It combines an AI legal chat assistant, document generation, lawyer discovery, Know Your Rights (KYR) content, consultation booking, and anonymous legal-story sharing into one browser-based product.

The platform targets citizens needing an accessible first step into the legal system: understanding rights, drafting first-pass legal documents, finding a lawyer, or reading and sharing legal experiences. AI output is framed as general information rather than legal advice, and several flows gate actions behind Supabase authentication.

| Feature Area        | Routes                            | Key Components                            |
|---------------------|-----------------------------------|-------------------------------------------|
| AI Legal Chat       | /chat, /chat/[id]                 | chat-interface.tsx, lib/puter-response.ts |
| Document Generation | /documents, /documents/[template] | document-generator.tsx                    |
| Lawyer Marketplace  | /lawyers, /lawyers/[id]           | lawyer-directory.tsx, lawyer-profile.tsx  |
| Know Your Rights    | /kyr, /kyr/[category]/...         | kyr-browse.tsx, kyr-article-view.tsx      |
| Community Stories   | /stories, /stories/[id]           | stories-directory.tsx, story-view.tsx     |
| User Dashboard      | /dashboard, /dashboard/profile    | user-dashboard.tsx, user-settings.tsx     |

Current status: PROTOTYPE — indicated by metadata.generator = "v0.app", overlapping schema families, disabled TypeScript build errors, and no test suite.

2

Architecture & Design

The application is a Next.js App Router monolith: route pages, route handlers, shared libraries, and UI components all live in one repository and deploy together. There are no independently deployed microservices, queues, or worker processes.

Design Patterns

| Pattern                | Description                                                          |
|------------------------|----------------------------------------------------------------------|
| App Router composition | app/**/page.tsx files own route-level orchestration                  |
| Thin route handlers    | API files validate requests, check auth, and do one focused task     |
| Client / server split  | Pages fetch data server-side; interactive widgets run in the browser |
| RLS-first access       | Supabase Row Level Security is the primary authorization boundary    |
| View-model mapping     | Pages reshape DB rows before passing them to presentation components |

Data Flow

Browser → middleware.ts → App Router page / API route → Supabase → rendered UI. For AI features, the browser also calls Puter.js, then persists results back to Supabase.

| Layer       | Component              | Responsibility                                            |
|-------------|------------------------|-----------------------------------------------------------|
| Entry       | User Browser           | React client, Puter.js AI, Supabase browser client        |
| Middleware  | middleware.ts          | Session refresh, auth-gated route protection              |
| Routing     | App Router pages       | Server components, data fetching, layout composition      |
| Server data | lib/supabase/server.ts | Cookie-scoped Supabase client for SSR                     |
| API         | app/api/** routes      | Thin handlers: validate → auth → Supabase → respond       |
| Persistence | Supabase Postgres      | Postgres + RLS + Auth, hosted by Supabase                 |
| AI          | Puter.js (CDN)         | Browser-loaded AI service for chat + document enhancement |

AI Chat Sequence

| Step | Actor    | Action                                              |
|------|----------|-----------------------------------------------------|
| 1    | User     | Types legal question in chat-interface.tsx          |
| 2    | Client   | Loads or creates a chat_session in Supabase         |
| 3    | Client   | Saves user message to chat_messages                 |
| 4    | Client   | Calls window.puter.ai.chat(prompt + context)        |
| 5    | Puter.js | Returns AI reply object                             |
| 6    | Client   | Calls extractPuterResponseText() to normalize shape |
| 7    | Client   | Saves assistant message to chat_messages            |

8      UI      Renders full conversation thread

### Auth Middleware Guard (from lib/supabase/middleware.ts)

```
if (!user && (
  request.nextUrl.pathname.startsWith("/dashboard") ||
  request.nextUrl.pathname.startsWith("/chat") ||
  request.nextUrl.pathname.startsWith("/documents") ||
  request.nextUrl.pathname.startsWith("/lawyers")
)) {
  const url = request.nextUrl.clone()
  url.pathname = "/auth/login"
  return NextResponse.redirect(url)
}
```

3

Tech Stack

| Area               | Package / Tool                                   | Version            |
|--------------------|--------------------------------------------------|--------------------|
| Runtime            | Node.js                                          | >= 18              |
| App framework      | next, react, react-dom                           | latest             |
| Language           | typescript                                       | ^5                 |
| Auth / data        | @supabase/ssr, @supabase/supabase-js             | latest             |
| UI primitives      | @radix-ui/*                                      | 1.1.x – 2.2.x      |
| Styling            | tailwindcss, tailwindcss-animate, tw-animate-css | ^4.1.9             |
| Motion / charts    | framer-motion, recharts                          | latest, 2.15.4     |
| Forms / validation | react-hook-form, @hookform/resolvers, zod        | ^7.60, ^3.10, 3.25 |
| Icons / themes     | lucide-react, next-themes, geist                 | mixed              |
| Date / OTP         | date-fns, react-day-picker, input-otp            | mixed              |
| AI (external)      | Puter.js                                         | v2 (CDN)           |

Language Distribution (approximate)

| Language         | Share | Primary use                                        |
|------------------|-------|----------------------------------------------------|
| TypeScript / TSX | ~80%  | All React components, API routes, utilities, hooks |
| SQL              | ~12%  | Supabase schema scripts, RLS policies, seed data   |
| Markdown         | ~8%   | Documentation files (README, DOCUMENTATION, etc.)  |

4

Project Structure

```
nyaya-ai-main/
├── app/           Next.js App Router: pages, layouts, API routes
├── components/    Feature components and reusable UI
│   ├── ui/       Radix/shadcn-style primitives
│   ├── hooks/    Shared React hooks
│   ├── lib/      Supabase clients, i18n, utilities, AI helpers
│   ├── public/   Static files and exported docs
│   ├── scripts/  Supabase SQL schema, seed, and policy scripts
│   ├── styles/   Legacy/global CSS assets
│   ├── middleware.ts  Auth/session refresh middleware
│   ├── next.config.mjs  Next.js config
│   ├── tsconfig.json   TypeScript config
│   ├── postcss.config.mjs  Tailwind/PostCSS config
│   ├── components.json  shadcn/ui config
│   └── nyayaAI-main/    Nested duplicate snapshot (secondary)
```

Key Module Reference

| File / Path                         | Purpose                                                                   |
|-------------------------------------|---------------------------------------------------------------------------|
| app/layout.tsx                      | Root HTML shell, fonts, theme provider, i18n provider, Puter.js injection |
| middleware.ts                       | Session refresh and route protection                                      |
| lib/supabase/server.ts              | Server-side Supabase client factory                                       |
| lib/supabase/client.ts              | Browser Supabase client factory                                           |
| lib/puter-response.ts               | Normalizes multiple AI response shapes into plain text                    |
| lib/i18n/translations.ts            | Translation dictionary for en, hi, mr, te                                 |
| components/chat-interface.tsx       | Authenticated AI chat UI and session persistence                          |
| components/document-generator.tsx   | Template filling, preview, AI enhancement, save/download                  |
| components/lawyer-directory.tsx     | Lawyer search, filter, sort directory                                     |
| components/consultation-booking.tsx | Consultation request and payment simulation flow                          |
| components/kyr-browse.tsx           | KYR category browser                                                      |
| components/kyr-article-view.tsx     | Single article view, sharing, bookmarking                                 |
| components/stories-directory.tsx    | Legal story feed and filters                                              |
| components/user-dashboard.tsx       | Consultation dashboard and review history                                 |
| components/user-settings.tsx        | Settings, profile, notifications, account deletion                        |

5

Setup & Installation

Prerequisites

- Node.js 18 or newer
- npm (package-lock.json present in root)
- Supabase project with URL and anon key
- Internet access for Puter.js CDN (<https://js.puter.com/v2/>)

Local Setup Steps

| Step | Action                    | Command / Note                                             |
|------|---------------------------|------------------------------------------------------------|
| 1    | Clone the repository      | git clone <repo> && cd nyaya-ai-main                       |
| 2    | Install dependencies      | npm install                                                |
| 3    | Copy environment template | cp env.example .env.local                                  |
| 4    | Set Supabase keys         | Edit .env.local with your project URL and anon key         |
| 5    | Run SQL scripts           | Execute numbered scripts in Supabase SQL editor (in order) |
| 6    | Start development server  | npm run dev                                                |
| 7    | Open in browser           | http://localhost:3000                                      |

Environment Variables

| Variable                              | Required  | Description                     | Example                         |
|---------------------------------------|-----------|---------------------------------|---------------------------------|
| NEXT_PUBLIC_SUPABASE_URL              | Yes       | Supabase project URL            | https://xyz.supabase.co         |
| NEXT_PUBLIC_SUPABASE_ANON_KEY         | Yes       | Public Supabase anon key        | eyJhbGciOi...                   |
| SUPABASE_SERVICE_ROLE_KEY             | Docs only | Service role key (legacy setup) | service-key                     |
| NEXT_PUBLIC_APP_URL                   | Docs      | Base URL for the app            | http://localhost:3000           |
| NEXT_PUBLIC_DEV_SUPABASE_REDIRECT_URL | Optional  | Signup email redirect override  | http://localhost:3000/dashboard |

6

Configuration

| File               | What it controls                                                    |
|--------------------|---------------------------------------------------------------------|
| next.config.mjs    | Image optimization, TypeScript build errors, lint behavior          |
| tsconfig.json      | TypeScript strictness, path aliases (@/* → root), module resolution |
| postcss.config.mjs | Tailwind/PostCSS processing pipeline                                |
| components.json    | shadcn/ui component generation conventions                          |
| middleware.ts      | Route protection and session refresh                                |
| app/layout.tsx     | Theme provider, i18n provider, fonts, Puter.js script tag           |

Notable Config Keys

| Key                          | Effect                                              |
|------------------------------|-----------------------------------------------------|
| typescript.ignoreBuildErrors | Suppresses TypeScript build failures — HIGH RISK    |
| eslint.ignoreDuringBuilds    | Present but unsupported by active Next.js toolchain |
| images.unoptimized           | Disables Next.js image optimization                 |
| paths (@/*)                  | tsconfig maps @/* to repo root for clean imports    |
| suppressHydrationWarning     | Used on <html> and <body> in root layout            |

No explicit feature-flag system was found in the active root workspace. Theme defaults to light mode via next-themes. Dev uses .env.local; signup redirects fall back to browser origin + /dashboard.



7

Database Schema

The active app code primarily uses the following tables. Multiple overlapping schema families exist in the SQL scripts (case\_stories, legal\_articles, user\_bookmarks, etc.) — these are historical and the newer families below are canonical.

| Table                         | Key Columns                                                               | Used By                |
|-------------------------------|---------------------------------------------------------------------------|------------------------|
| users                         | id, email, full_name, phone, location, profession                         | settings, profile page |
| chat_sessions                 | id, user_id, title, created/updated_at                                    | chat, history          |
| chat_messages                 | id, session_id, content, role, created_at                                 | chat, AI persistence   |
| document_templates            | id, title, category, slug, fields, template_content                       | document generator     |
| generated_documents           | id, user_id, template_id, title, content                                  | document history       |
| kyr_articles                  | id, slug, category, summary, content, read_time                           | KYR browser            |
| bookmarks                     | id, user_id, article_id, created_at                                       | KYR bookmarking        |
| lawyers                       | id, full_name, email, experience_years, practice_areas, rating, directory | directory, profile     |
| lawyer_specializations        | id, name, description, icon                                               | lookup table           |
| lawyer_specialization_mapping | id, lawyer_id, specialization_id                                          | many-to-many bridge    |
| consultation_requests         | id, user_id, lawyer_id, subject, status                                   | booking lifecycle      |
| lawyer_reviews                | id, user_id, lawyer_id, consultation_request_id, rating                   | reviews                |
| legal_stories                 | id, user_id, title, case_type, is_anonymous, is_approved                  | community stories      |
| story_comments                | id, story_id, user_id, content, is_anonymous                              | story comments         |
| story_likes                   | id, story_id, user_id, created_at                                         | story reactions        |
| story_categories              | id, name, description, icon, color                                        | category lookup        |
| story_category_mapping        | id, story_id, category_id                                                 | story/category bridge  |

Entity Relationships

| Parent             | Relationship | Child                                |
|--------------------|--------------|--------------------------------------|
| users              | 1 → N        | chat_sessions                        |
| chat_sessions      | 1 → N        | chat_messages                        |
| users              | 1 → N        | generated_documents                  |
| document_templates | 1 → N        | generated_documents                  |
| users              | 1 → N        | bookmarks                            |
| kyr_articles       | 1 → N        | bookmarks                            |
| lawyers            | N ↔ N        | lawyer_specializations (via mapping) |
| users              | 1 → N        | consultation_requests                |
| lawyers            | 1 → N        | consultation_requests                |

|                       |       |                             |
|-----------------------|-------|-----------------------------|
| consultation_requests | 1 → N | lawyer_reviews              |
| legal_stories         | 1 → N | story_comments              |
| legal_stories         | 1 → N | story_likes                 |
| story_categories      | N ↔ N | legal_stories (via mapping) |

8

API Reference

| Method | Path                   | Auth     | Returns                   | Notes                                      |
|--------|------------------------|----------|---------------------------|--------------------------------------------|
| POST   | /api/chat              | Required | plain text                | Sends prompt to Puter.js, returns AI reply |
| POST   | /api/chat/start        | Required | { conversationId }        | Creates/reuses lawyer conversation         |
| POST   | /api/enhance-document  | Required | plain text                | Rules-based string enhancer                |
| POST   | /api/documents/enhance | Required | { enhanced_content }      | Puter.js AI enhancement                    |
| GET    | /api/bookmarks         | Required | { bookmarks: [...] }      | Lists user bookmarks with article data     |
| POST   | /api/bookmarks         | Required | { action: added removed } | Toggles bookmark for KYR article           |

Example: POST /api/chat

```
// Request body
{ "message": "What are my tenant rights in Mumbai?", "sessionId": "..." }

// Response (plain text)
Under Indian law, tenants generally have rights related to notice,
habitability, and refund of security deposit...

// Behavior
1. Validate message field
2. Load authenticated user via Supabase
3. Fetch last 10 chat_messages for context
4. Build legal system prompt (Indian law focus)
5. Call global.puter.ai.chat(systemPrompt)
6. Return reply.output_text
```

Auth Flow (all API routes)

| Step | Action                                              |
|------|-----------------------------------------------------|
| 1    | Client calls API endpoint with session cookie       |
| 2    | API route calls supabase.auth.getUser()             |
| 3    | If no user: return 401 Unauthorized                 |
| 4    | If user: proceed with Supabase query (respects RLS) |
| 5    | Return JSON or plain text response to client        |

## 9

## Key Algorithms & Business Logic

### Chat Session Lifecycle

File: chat-interface.tsx

- 1. Load user sessions on mount
- 2. Create new chat\_sessions row if needed
- 3. Append user message locally (optimistic update)
- 4. Persist user message to chat\_messages
- 5. Wait for window.puter.ai to be available
- 6. Call ai.chat(prompt + last-N context)
- 7. Normalize response with extractPuterResponseText()
- 8. Save assistant reply to chat\_messages
- 9. Re-render conversation thread

### Document Generation

File: document-generator.tsx

- 1. Load document\_templates row by slug
- 2. Render form fields from template.fields (JSONB)
- 3. Replace {fieldId} placeholders with form values
- 4. Preview generated text in real-time
- 5. Optionally POST to /api/enhance-document or /api/documents/enhance
- 6. On save/download: insert into generated\_documents

### Bookmark Toggling

File: kyr-article-view.tsx / api/bookmarks

- 1. User clicks bookmark button
- 2. If not logged in: prompt login
- 3. POST /api/bookmarks with { articleId }
- 4. API checks if bookmarks row exists for (user\_id, article\_id)
- 5. If exists: DELETE row, return { action: 'removed' }
- 6. If absent: INSERT row, return { action: 'added' }
- 7. UI updates button state from response

### Filtering & Sorting (lawyer + story directories)

File: lawyer-directory.tsx, stories-directory.tsx

- 1. All filtering runs in useMemo() to avoid redundant re-renders
- 2. Lawyer filters: specialization, location, experience, availability, language
- 3. Sort options: rating, experience, fee
- 4. Story filters: case\_type, outcome, location
- 5. Performance note: no server-side pagination yet — all rows loaded upfront

Consultation booking creates a request then simulates payment by setting status = 'paid'. WARNING: the inspected SQL schema does not include 'paid' in the status CHECK constraint. This path needs verification before production use.

10

Testing

No automated test framework is configured in the active root workspace. Jest, Vitest, Playwright, and Cypress were not found. The only verification path is manual.

| Test Type         | Status         | Notes                                |
|-------------------|----------------|--------------------------------------|
| Unit tests        | Not configured | No Jest/Vitest setup found           |
| Integration tests | Not configured | No integration runner found          |
| End-to-end tests  | Not configured | No Playwright/Cypress found          |
| Manual testing    | Active         | Currently the only verification path |

Recommended coverage priorities when tests are added:

- Auth: signup, login, logout, session refresh, protected route redirect
- Chat: session creation, message persistence, Puter.js response normalization
- Document generation: template loading, placeholder substitution, save flow
- Lawyer consultation booking: request creation, status transitions
- Bookmark toggling: add and remove idempotency
- KYR article browsing: filter and search flows

11

## Deployment

No CI/CD pipeline or deployment config (.github/workflows, vercel.json, Dockerfile) was found in the active root workspace. Deployment is currently manual.

### Build Commands

```
npm run build
npm run start
```

| Target       | Fit       | Notes                                                   |
|--------------|-----------|---------------------------------------------------------|
| Vercel       | Excellent | Natural fit for Next.js App Router with edge middleware |
| Supabase     | Required  | Hosts Postgres + Auth; not optional                     |
| Puter.js CDN | External  | Loaded from https://js.puter.com/v2/ at runtime         |
| Docker       | Possible  | No Dockerfile present; would need to be created         |

### Rollback Strategy

- No automated rollback configured
- Manual: redeploy a previous Git commit on hosting platform
- Schema rollback is not automated — apply DB changes cautiously
- Recommended: version all SQL scripts and test in staging before production

12

Known Issues & Limitations

| Issue                        | Severity | Evidence / Location                                                                |
|------------------------------|----------|------------------------------------------------------------------------------------|
| Duplicate schema families    | HIGH     | users vs user_profiles, bookmarks vs user_bookmarks, legal_stories vs case_stories |
| Disabled build checks        | HIGH     | typescript.ignoreBuildErrors + eslint.ignoreDuringBuilds in next.config.mjs        |
| Unsupported config key       | MEDIUM   | Next.js reports eslint.ignoreDuringBuilds as unsupported                           |
| No automated tests           | HIGH     | No test runner or npm test script found                                            |
| Server-side Puter usage      | HIGH     | api/chat/route.ts references global.puter server-side (browser-only library)       |
| Consultation status mismatch | MEDIUM   | UI sets status='paid'; schema CHECK constraint omits this value                    |
| Story flow drift             | MEDIUM   | Components use legal_stories; older scripts define case_stories                    |
| Profile flow drift           | MEDIUM   | profile/page.tsx writes users; settings/page.tsx reads user_profiles               |

Issue Priority Matrix

| Issue                                      | Effort | Impact    | Quadrant             |
|--------------------------------------------|--------|-----------|----------------------|
| Fix unsupported config key                 | Low    | High      | Quick win — do first |
| Remove duplicate docs drift                | Low    | High      | Quick win            |
| Unify profile tables (users/user_profiles) | High   | Very High | Major project        |
| Add automated test suite                   | High   | Very High | Major project        |
| Resolve server-side Puter.js usage         | Medium | High      | High priority        |
| Fix consultation status constraint         | Medium | High      | High priority        |



13

Contributing Guide

No formal contributor guide file exists. Conventions below are inferred from repo shape.

| Area            | Convention                                                              |
|-----------------|-------------------------------------------------------------------------|
| Branch naming   | Suggested: feature/<name>, fix/<name>, docs/<name>                      |
| Commit messages | Descriptive (no strict conventional-commit policy found)                |
| PR process      | Smallest coherent change, manual smoke test, screenshots for UI changes |
| Code style      | TypeScript strict mode enabled (but build errors suppressed by config)  |
| Styling system  | Tailwind CSS exclusively                                                |
| Component split | use 'use client' directive when browser APIs or hooks are needed        |
| Testing         | Explicitly document manual verification performed in PR description     |

14

Changelog / Version History

No CHANGELOG.md exists. History inferred from repo shape, SQL script names, and README files.

| Period       | Milestone         | Details                                                                |
|--------------|-------------------|------------------------------------------------------------------------|
| 2024 — Early | Initial prototype | Core Next.js + Supabase scaffolding, v0-generated app shell            |
| 2024 — Mid   | KYR content       | Article browsing, Supabase-backed content, kyr_articles table          |
| 2025 — Q1    | Lawyer directory  | Lawyer profiles, consultation booking, reviews, specialization mapping |
| 2025 — Q2    | Community stories | legal_stories, story_comments, story_likes, story_categories           |
| 2026         | AI chat + docs    | Puter.js integration, document generation, history views, i18n         |

15

Glossary

| Term                     | Definition                                                                                             |
|--------------------------|--------------------------------------------------------------------------------------------------------|
| KYR                      | Know Your Rights. The rights-education area under /kyr, /kyr/[category], /kyr/article/[slug].          |
| RLS                      | Row Level Security in Supabase/Postgres. The main authorization layer for most tables.                 |
| Puter.js                 | A browser-loaded AI service used for chat-style responses and document enhancement.                    |
| sessionId                | The identifier for an AI chat session in chat_sessions, or a lawyer chat conversation.                 |
| conversationId           | The lawyer-chat conversation identifier returned by /api/chat/start.                                   |
| slug                     | URL-friendly text used in paths and content lookups (e.g., kyr_articles.slug).                         |
| case_type                | The category of a legal story, such as 'Family Law' or 'Consumer Rights'.                              |
| consultation_request     | A user-to-lawyer request with subject, description, preferred time, and status.                        |
| practice_areas           | An array column on lawyers representing specialties shown in the directory UI.                         |
| specialization mapping   | The bridge table linking lawyers to one or more specialization rows.                                   |
| is_approved              | Moderation flag for KYR articles and stories; only approved content is public.                         |
| is_anonymous             | Privacy flag for stories and comments to hide author identity.                                         |
| NEXT_PUBLIC_*            | Environment variables exposed to browser-side code by Next.js convention.                              |
| updateSession            | Helper in lib/supabase/middleware.ts that refreshes cookies and enforces redirects.                    |
| extractPuterResponseText | Helper in lib/puter-response.ts that normalizes AI response shapes into plain text.                    |
| generated_documents      | Saved, user-owned legal documents produced by the template generator.                                  |
| legal_stories            | The currently active story table used by story pages and the story UI.                                 |
| user_profiles            | Table referenced by some settings code; not in numbered setup scripts. Treat as unresolved dependency. |
| paid                     | A consultation status set by booking UI, though not in the inspected schema CHECK constraint.          |

Mental model for new engineers: Next.js App Router on the front end — Supabase on the back end — Puter.js for AI — SQL scripts as source of truth for the data model. The main risk area is schema drift between SQL scripts and components.

End of Nyaya.ai Technical Documentation